INFORME DE LABORATORIO 1

Aplicación bancaria web con backend Spring Boot y frontend React

Universidad	Universidad de Antioquia
Facultad	Facultad de Ingeniería
Asignatura	Arquitectura de Software
Profesor	Diego José Luis Botia Valderrama
Fecha	13 de marzo de 2026

Integrantes

- Santiago Palacio Cárdenas
- Sarai Restrepo Rodríguez
- Juan Pablo Herrera Jaramillo
- Jimena Muñoz Gómez

Repositorio GitHub: https://github.com/spalacioc05/lab12026p

1. Introducción

En el presente laboratorio se desarrolló una aplicación web orientada a la simulación de transacciones bancarias, con el propósito de aplicar conceptos fundamentales de arquitectura por capas, desarrollo backend con Spring Boot, persistencia en bases de datos relacionales y construcción de interfaces web para el consumo de servicios REST. El enunciado del laboratorio planteaba la creación de una solución compuesta por dos partes principales: un backend encargado de exponer una API para gestionar clientes y transferencias, y un frontend que permitiera interactuar con dichas funcionalidades mediante una interfaz amigable.

Para cumplir este objetivo, se implementó un servicio backend en Spring Boot conectado a una base de datos MySQL, utilizando una organización por capas basada en controladores, servicios, repositorios, entidades y DTOs. Esta arquitectura permitió separar responsabilidades y mantener un diseño más limpio, escalable y mantenible, en concordancia con la estructura propuesta por el laboratorio. En el documento base también se planteaba explícitamente esta arquitectura multicapa y la separación entre modelo, repositorio, servicio y controlador.

Adicionalmente, se construyó un frontend en React para consumir las rutas definidas por el backend. Esta interfaz incluyó las tres vistas mínimas exigidas: consulta de clientes, realización de transferencias entre cuentas y visualización del histórico de transacciones por cliente, tal como fue solicitado en la entrega obligatoria del laboratorio.

Durante el desarrollo también se realizaron pruebas funcionales mediante Postman, tanto para verificar la creación de clientes como para validar la consulta de información y el correcto funcionamiento del proceso de transferencia. La colección de pruebas utilizada incluyó peticiones para crear cliente, listar clientes, buscar por id, buscar por número de cuenta, transferir dinero y consultar el historial de movimientos por cuenta.

En conjunto, este laboratorio permitió integrar conocimientos de backend, frontend, bases de datos y pruebas de servicios, consolidando una solución funcional que simula un flujo bancario básico y evidencia la importancia de una arquitectura bien definida desde las primeras etapas del desarrollo.

2. Objetivos

Objetivo general

Desarrollar una aplicación web bancaria básica, compuesta por un backend en Spring Boot y un frontend en React, que permita gestionar clientes, realizar transferencias entre cuentas y consultar el historial de transacciones, aplicando principios de arquitectura por capas y consumo de servicios REST.

Objetivos específicos

- Configurar un proyecto backend en Spring Boot con Maven y Java 17.
- Conectar la aplicación a una base de datos MySQL para almacenar clientes y transacciones.

- Organizar el backend mediante una arquitectura por capas que incluya controladores, servicios, repositorios, entidades y DTOs.
- Implementar reglas de negocio para validar transferencias bancarias, incluyendo existencia de cuentas, saldo suficiente y restricciones de montos.
- Construir un frontend en React que consuma las rutas REST expuestas por el backend.
- Desarrollar las tres vistas mínimas requeridas en el laboratorio: consultar clientes, realizar transferencias y visualizar histórico de transacciones.
- Realizar pruebas funcionales con Postman para comprobar el comportamiento del sistema.
- Presentar una solución integrada, comprensible y alineada con los requerimientos académicos del laboratorio.

3. Herramientas de software empleadas

Para el desarrollo del laboratorio se utilizaron diversas herramientas y tecnologías tanto en backend como en frontend. El documento del laboratorio proponía como base herramientas como Spring Boot, Java, NodeJS, React, Axios y Maven.

Herramienta	Categoría	Uso principal
Spring Boot	Backend	Se utilizó para construir el backend de la aplicación y exponer la API REST necesaria para la gestión de clientes y transacciones.
Java 17	Lenguaje	Fue el lenguaje base del backend, escogido por su compatibilidad con Spring Boot y por ser una de las versiones sugeridas en el laboratorio.
Maven	Build	Se utilizó para la gestión de dependencias, compilación y ejecución del proyecto backend.
MySQL	Base de datos	Fue el sistema gestor de bases de datos utilizado para almacenar la información persistente del sistema, específicamente clientes y transacciones.
DBeaver	Cliente DB	Se empleó como cliente gráfico para conectarse a MySQL, crear la base de datos, inspeccionar tablas y visualizar registros. Su uso resultó conveniente por su interfaz clara y facilidad para administrar la información almacenada.
React + Vite	Frontend	Se utilizó para construir el frontend web encargado de consumir la API REST y presentar la información

		de manera amigable al usuario.
Axios	Integración HTTP	Se utilizó para realizar las solicitudes HTTP desde el frontend hacia el backend.
Postman	Pruebas	Se utilizó para probar y validar los endpoints del backend antes de integrarlos al frontend. La colección de pruebas incluyó operaciones clave como crear cliente, listar clientes, buscar clientes y ejecutar transferencias.
Visual Studio Code	IDEA	Fue utilizado como entorno de desarrollo principal para editar tanto el frontend como parte del backend.
IntelliJ IDEA	IDEA	Puede considerarse como herramienta apropiada para el backend en proyectos Spring Boot, tal como también era sugerido en el laboratorio dentro de los IDE recomendados.
JSON	Formatos	Se utilizó como formato de intercambio de datos entre backend, frontend y pruebas con Postman.
XML	Formatos	Se tuvo presente dentro de la configuración estructural de algunos archivos del proyecto, especialmente en el entorno backend con Maven, donde *pom.xml* representa la configuración base de dependencias y build del proyecto. El laboratorio también dedicaba una sección a la estructura del archivo *pom.xml*.

4. Arquitectura propuesta

La solución desarrollada siguió una arquitectura cliente-servidor con separación clara entre frontend, backend y capa de persistencia. Esta propuesta coincide con el enfoque del laboratorio, que planteaba una organización por capas y mostraba además un diagrama donde se relacionaban frontend, controladores, servicios, repositorios, entidades, DTOs, mappers y base de datos.

De forma general, la arquitectura se compone de tres bloques principales:

Frontend

El frontend fue desarrollado en React y tiene la responsabilidad de presentar la información al usuario, capturar datos mediante formularios y consumir los servicios expuestos por el backend. Desde esta capa se implementaron las vistas obligatorias del laboratorio:

- consulta de clientes,
- transferencia de dinero entre cuentas,
- histórico de transacciones por cliente.

El frontend funciona como consumidor de la API REST y se comunica con el servidor mediante peticiones HTTP.

Backend

El backend se desarrolló con Spring Boot y fue estructurado por capas:

Controladores

Exponen los endpoints REST que reciben las solicitudes del cliente y devuelven respuestas en formato JSON.

Servicios

Contienen la lógica de negocio, como la validación de cuentas, la verificación de saldo disponible y el procesamiento de transferencias.

Repositorios

Gestionan la interacción con la base de datos mediante Spring Data JPA.

Entidades

Representan el modelo persistente de la aplicación, principalmente clientes y transacciones.

DTOs

Se utilizaron para transferir información entre capas y evitar exponer directamente las entidades al exterior. El laboratorio también justificaba expresamente el uso de DTOs y mappers como una buena práctica para desacoplar las entidades del dominio de la capa de exposición.

Base de datos

La persistencia se gestionó con MySQL, donde se almacenó la información de los clientes y las transacciones realizadas. Para su administración y consulta se utilizó DBeaver.

En conjunto, esta arquitectura permitió una solución modular, entendible y fácilmente ampliable. Además, favoreció la mantenibilidad del sistema, ya que cada componente cumple una responsabilidad concreta y bien delimitada.

5. Procedimiento

Análisis del enunciado del laboratorio

Inicialmente se revisó el documento guía del laboratorio, en el cual se establecía la necesidad de construir una aplicación bancaria que permitiera crear usuarios, realizar movimientos entre cuentas y consultar el histórico de transacciones. También se especificaba el uso de Spring Boot para el backend, React para el frontend y MySQL para la persistencia.

A partir de este análisis se definieron dos etapas principales de desarrollo: primero la construcción y validación del backend, y posteriormente la integración del frontend para consumir la API resultante.

Construcción del backend

Se levantó el proyecto backend con Spring Boot, Maven y Java 17. Posteriormente se configuró la conexión con la base de datos MySQL en el archivo de propiedades, adaptando el nombre de la base a banco2026 y usando las credenciales de conexión locales.

Se implementaron las entidades correspondientes a clientes y transacciones, siguiendo el modelo general propuesto por el laboratorio, en el cual se contemplan las tablas customers y transactions con atributos como número de cuenta, nombres, apellidos, saldo, cuentas de origen y destino, monto y marca temporal.

Después se organizaron las capas del proyecto en controladores, servicios, repositorios, entidades, DTOs, mappers y manejo de excepciones. Esta separación permitió encapsular la lógica de negocio y desacoplar la representación de datos usada en la API de la estructura interna persistente.

Implementación de reglas de negocio

Una vez definida la estructura general del backend, se implementaron las reglas de negocio esenciales para el sistema bancario. Entre ellas:

- validación de existencia de la cuenta emisora,
- validación de existencia de la cuenta receptora,
- verificación de saldo suficiente,
- restricción de montos menores o iguales a cero,
- restricción de transferencias a la misma cuenta,

- asignación automática de la fecha y hora de la transacción.

Estas reglas fueron importantes para garantizar coherencia en las operaciones y prevenir errores funcionales en la aplicación.

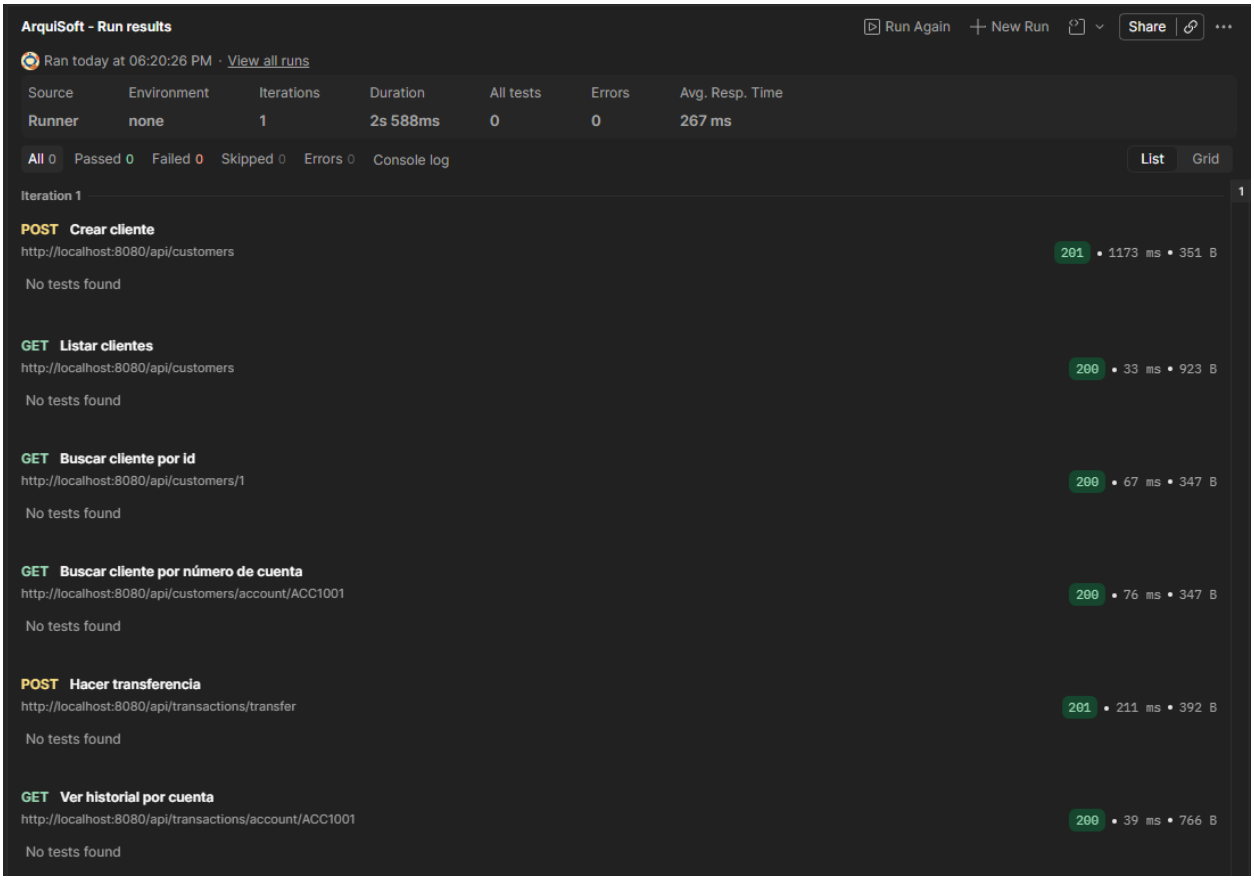
Configuración y uso de la base de datos

Se creó la base de datos en MySQL y se gestionó mediante DBeaver, herramienta que facilitó la visualización de tablas, registros y estructura general del modelo.

Se almacenaron clientes de prueba para verificar el comportamiento del sistema antes de conectar el frontend. Posteriormente se validó que las transferencias modificaran correctamente el saldo de las cuentas involucradas y que dichas operaciones quedaran reflejadas en el historial de transacciones.

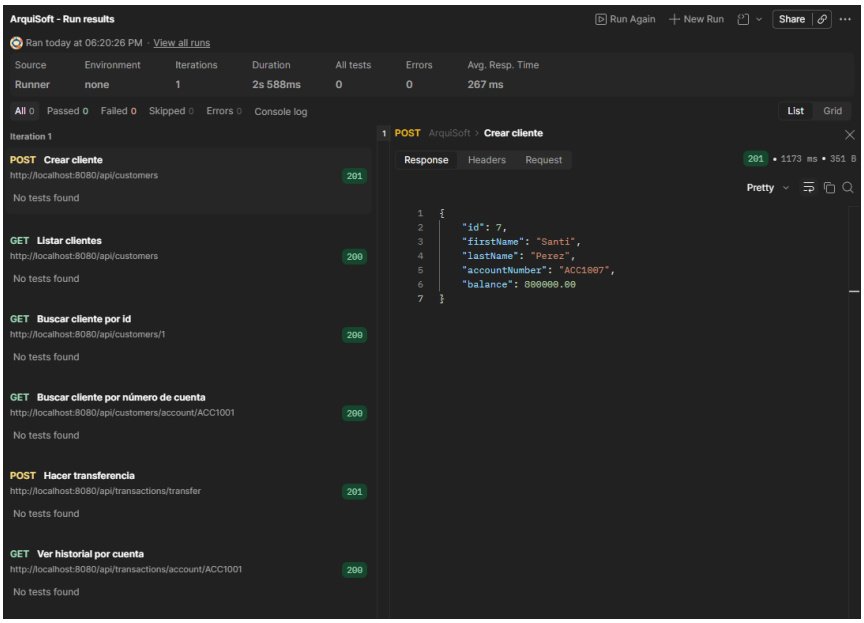
Pruebas del backend con Postman

Antes de integrar la interfaz de usuario, se realizaron pruebas con Postman para validar que los endpoints funcionaran correctamente. La colección utilizada incluyó las siguientes operaciones:



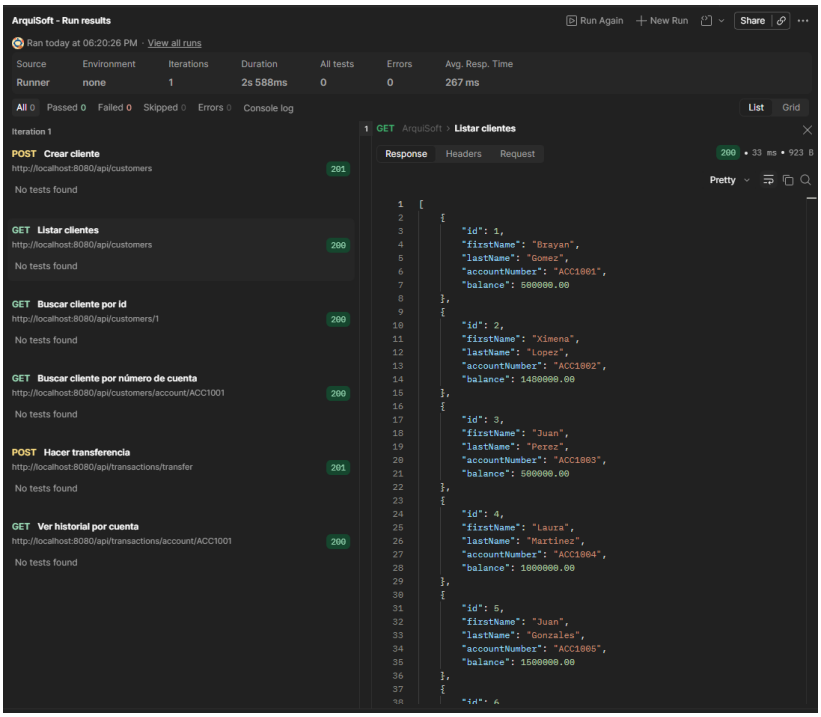
- **Crear cliente** mediante POST /api/customers, enviando un JSON con nombre, apellido, número de cuenta y saldo inicial.

  http://localhost:8080/api/customers



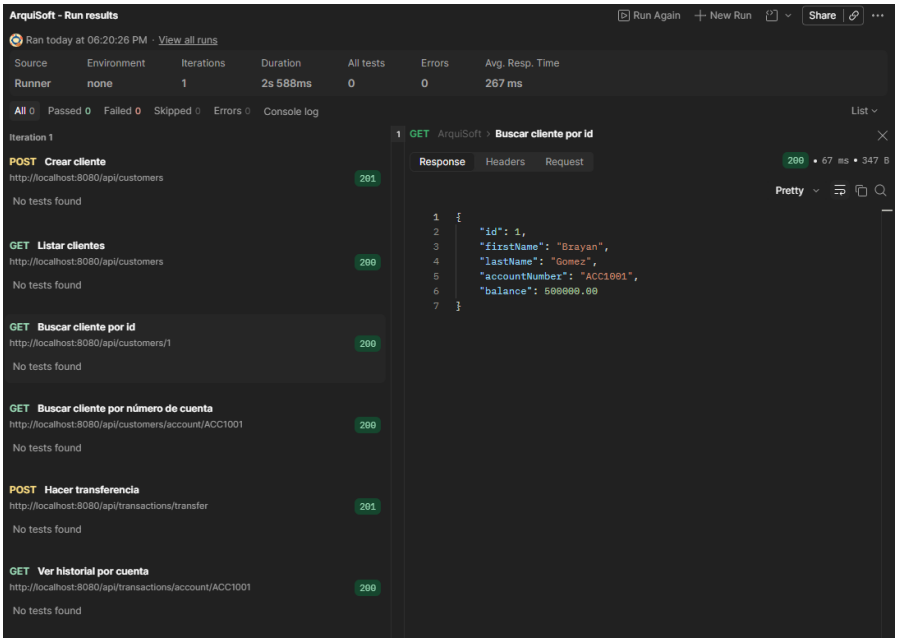
- **Listar clientes** mediante GET /api/customers.

  http://localhost:8080/api/customers



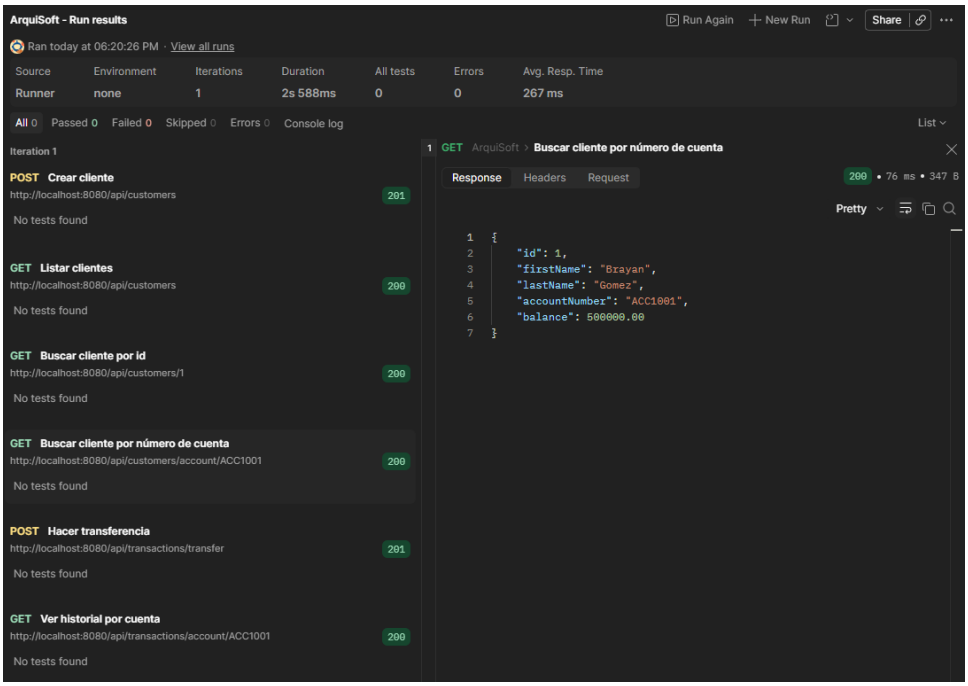
- **Buscar cliente por id** mediante GET /api/customers/{id}.

  http://localhost:8080/api/customers/1



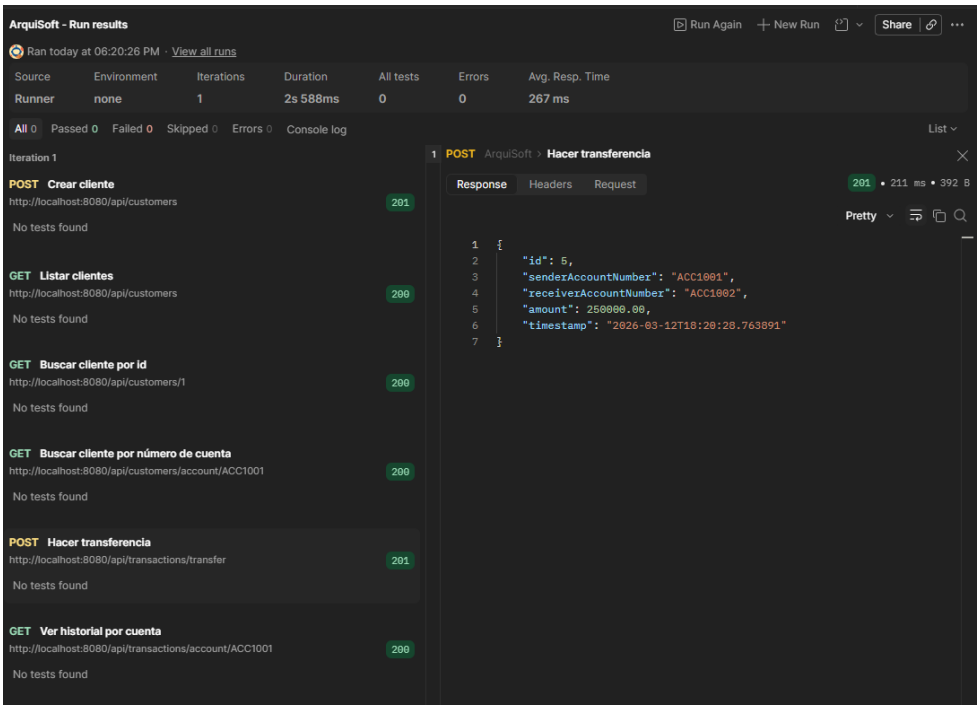
- **Buscar cliente por número de cuenta** mediante GET /api/customers/account/{accountNumber}.

  http://localhost:8080/api/customers/account/ACC1001



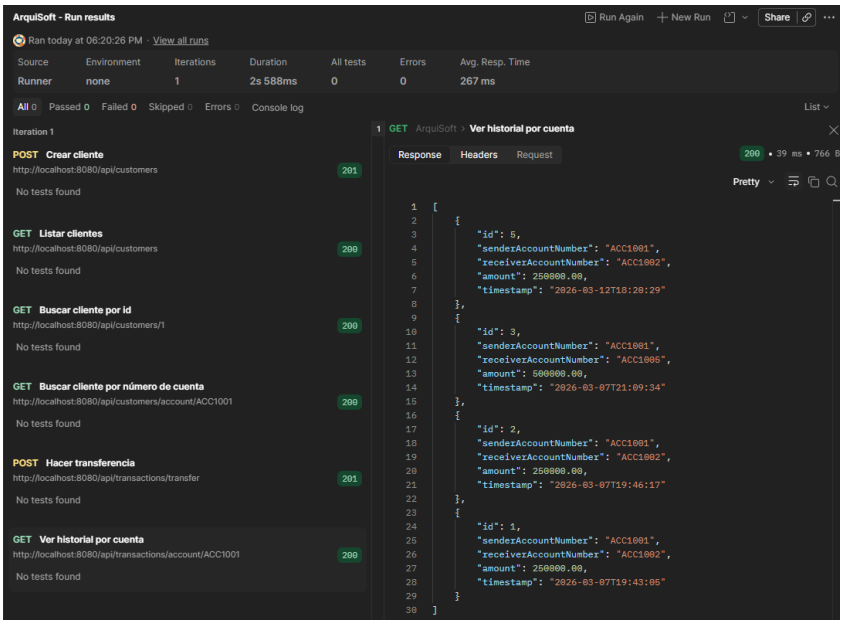
- **Hacer transferencia** mediante POST /api/transactions/transfer, enviando un JSON con cuenta origen, cuenta destino y monto.

  http://localhost:8080/api/transactions/transfer



- **Consultar historial por cuenta** mediante GET /api/transactions/account/{accountNumber}.

  http://localhost:8080/api/transactions/account/ACC1001



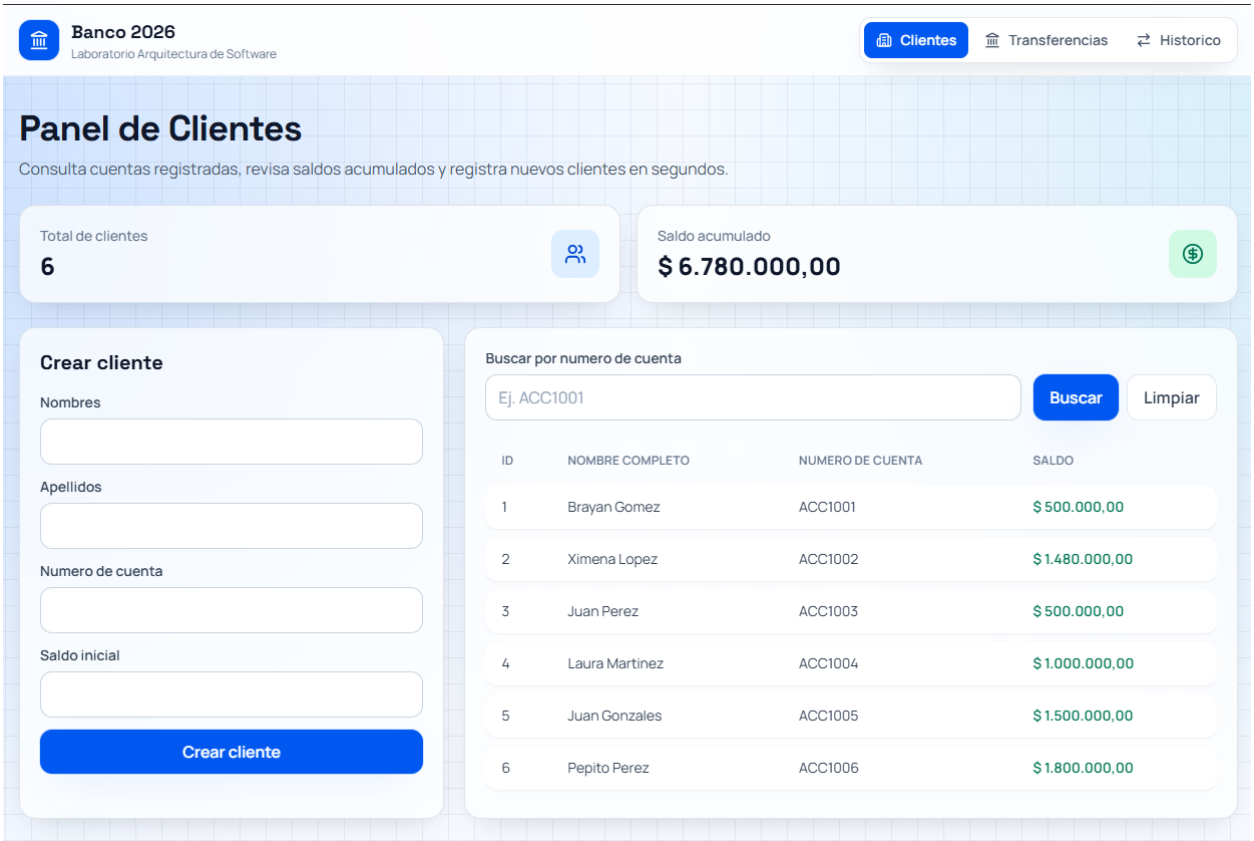
Como ejemplos de pruebas realizadas se emplearon cargas útiles como la creación del cliente Laura Martínez con cuenta ACC1004 y saldo 800000.00, así como una transferencia desde ACC1001 hacia ACC1002 por valor de 250000.00. Estas pruebas quedaron consignadas en la colección JSON de Postman utilizada en el proyecto.

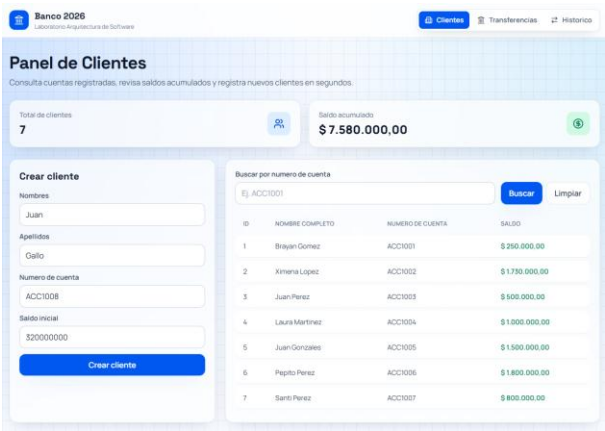
Desarrollo del frontend

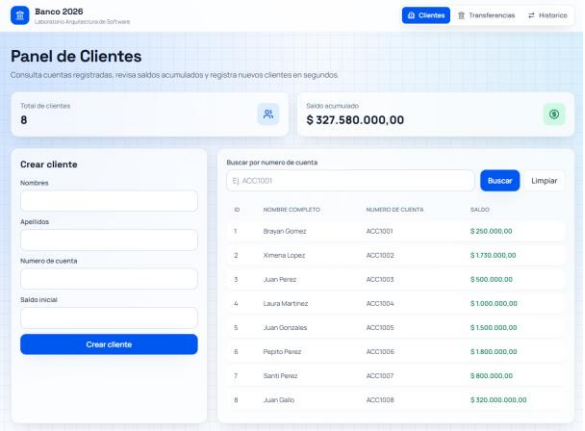
Con el backend ya funcional, se procedió a construir el frontend en React. Se creó una carpeta independiente para esta parte del proyecto y se organizó el código en componentes, páginas, servicios y utilidades, con el fin de mantener una estructura ordenada.

El frontend se diseñó para consumir directamente las rutas del backend y presentar una interfaz más agradable y moderna. Se implementaron como mínimo las tres vistas obligatorias solicitadas en el laboratorio: consultar clientes, realizar transferencias entre cuentas y visualizar el histórico de movimientos por cliente. Esta exigencia también aparece expresamente en el enunciado final del laboratorio.

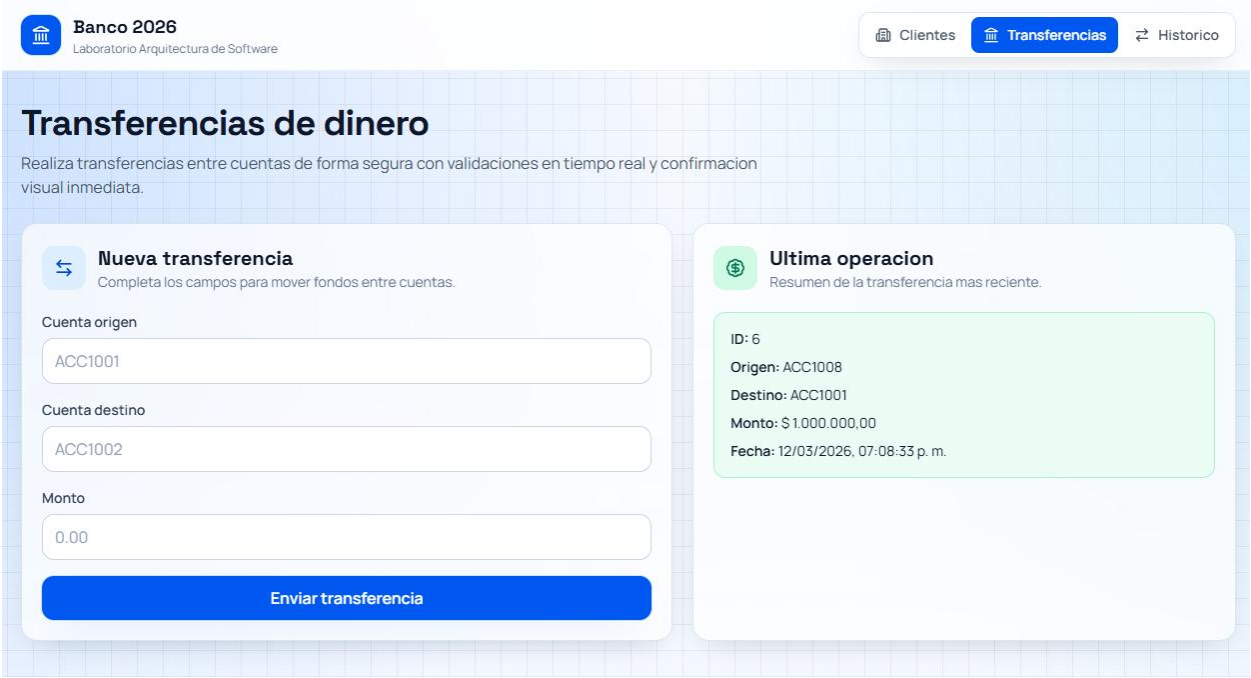
Vista de consulta de clientes



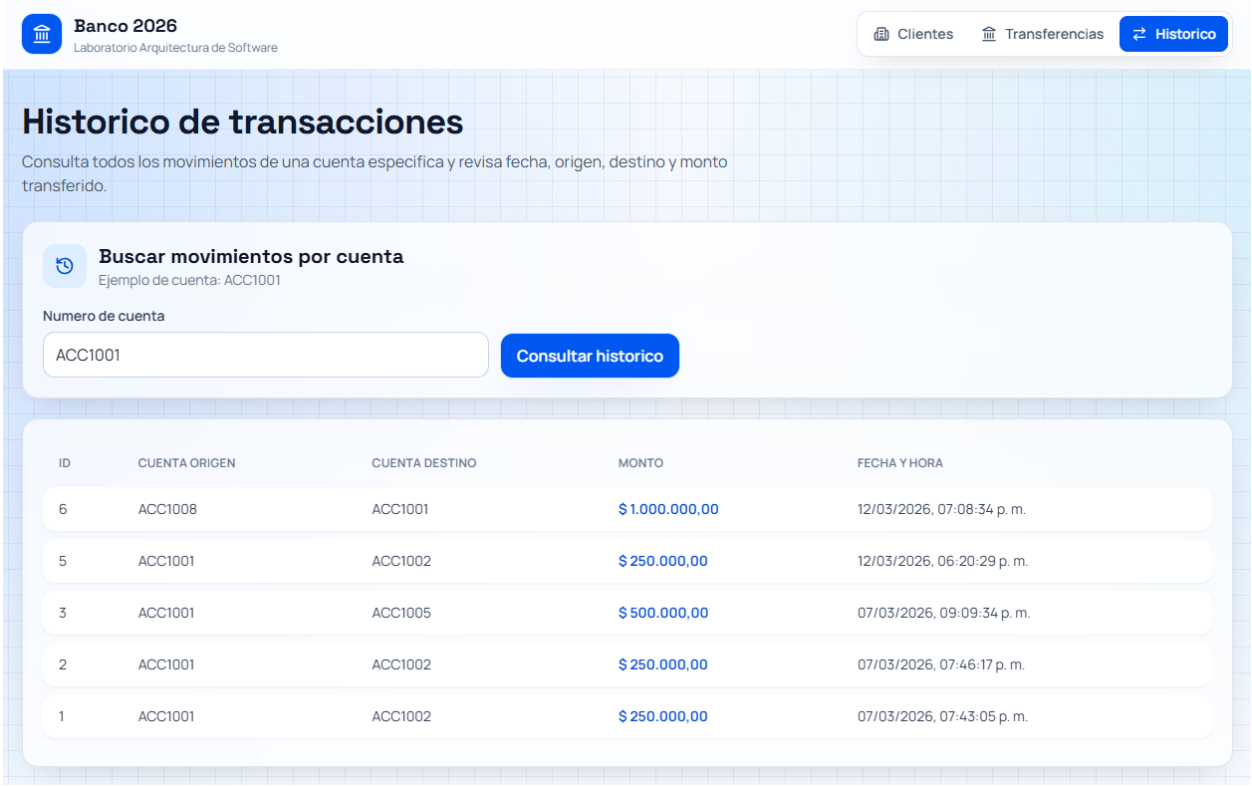




Vista de transferencia entre cuentas



Vista del historial de transacciones



Integración frontend-backend

Una vez creadas las vistas, se integró el frontend con el backend mediante solicitudes HTTP. El flujo general consistió en:

- el usuario accede a una vista del frontend,
- el frontend captura o solicita información,
- se realiza una petición al backend,
- el backend procesa la lógica correspondiente,
- la respuesta regresa en formato JSON,
- el frontend actualiza la interfaz con los resultados.

Esto permitió validar no solo el correcto funcionamiento de la API por separado, sino también la experiencia completa del sistema ya integrado.

7. Conclusiones

El desarrollo de este laboratorio permitió aplicar de forma práctica varios conceptos fundamentales de arquitectura de software, especialmente la organización por capas y la separación de responsabilidades entre frontend, backend y persistencia. Esta estructura resultó adecuada para construir una solución clara, mantenible y alineada con lo propuesto en el laboratorio.

La implementación del backend con Spring Boot demostró la utilidad de trabajar con controladores, servicios, repositorios, entidades y DTOs para construir APIs robustas y bien estructuradas. Asimismo, el uso de reglas de negocio permitió asegurar que las operaciones bancarias realizadas conservaran coherencia y evitaran errores comunes como transferencias inválidas o saldos insuficientes.

Por otra parte, el desarrollo del frontend en React complementó adecuadamente la solución, ya que hizo posible transformar una API funcional en una aplicación interactiva y visualmente más cercana a una experiencia real de usuario. La integración entre ambas capas permitió comprobar la importancia de diseñar servicios REST claros y fáciles de consumir.

También fue valioso el uso de herramientas de apoyo como DBeaver y Postman, ya que facilitaron tanto la administración de la base de datos como la validación sistemática de cada endpoint antes de la conexión con la interfaz.

En términos generales, el laboratorio cumplió su propósito formativo al permitir construir una aplicación completa, integrando distintas tecnologías y fortaleciendo competencias en desarrollo full stack, modelado por capas, pruebas de servicios y conexión con bases de datos.

8. Bibliografía

- Botia Valderrama, D. J. L. (2026). Laboratorio 1: Introducción a Spring Boot. Facultad de Ingeniería, Universidad de Antioquia.
- Spring. Documentación oficial de Spring Boot.
- React. Documentación oficial de React.
- Postman. Learning Center y documentación oficial.
- MySQL. Documentación oficial del motor de base de datos.
- DBeaver. Documentación oficial del cliente de base de datos.

9. Proyecto anexo en GitHub

En este apartado se debe incluir el enlace al repositorio del proyecto, acompañado de una breve descripción de su contenido. Se recomienda adjuntar la ruta principal, la estructura general del backend y frontend y, si aplica, instrucciones mínimas de ejecución.

Repositorio	https://github.com/spalacioc05/lab12026p